



AUDIT REPORT

June 2026

For



VIITUM

Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Informational Issues	14
1. Use Ownable2Step Instead of Ownable to Prevent Accidental Ownership Loss	14
2. Fix Floating Pragma to a Specific Compiler Version	15
Functional Tests	16
Threat Model	17
Automated Tests	25
Closing Summary & Disclaimer	26



Executive Summary

Project Name	VIITUM
Protocol Type	ERC20 Token
Project URL	https://viitum.io/
Overview	VIITUM is an ERC20 token (symbol: \$VIIT, name: VIITUM) built on OpenZeppelin Contracts ^5.4.0. The token supports standard transfers, burn functionality, gasless approvals via ERC20Permit, owner-controlled minting, and a custom batchTransfer function enabling token distribution to multiple recipients in a single transaction.
Audit Scope	The scope of this Audit was to analyze the VIITUM Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Peculiber/viitum/tree/main/backend/src/contracts
Branch	main
Contracts in Scope	Backend/src/contracts/BSToken.sol backend/src/contracts/TronToken.sol backend/src/contracts/erc20.sol
Commit Hash	34337726b4c1ba6a242317ed12fd0ad0bcde8942
Language	Solidity
Blockchain	Ethereum, Tron, BSC
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th June 2026
Updated Code Received	23rd June 2026
Review 2	23rd June 2026
Fixed In	34337726b4c1ba6a242317ed12fd0ad0bcde8942



Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	0 (0%)
Low	0 (0%)
Informational	2 (100.00%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	2



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Use Ownable2Step Instead of Ownable to Prevent Accidental Ownership Loss	Informational	Resolved
2	Fix Floating Pragma to a Specific Compiler Version	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level



Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Use Ownable2Step Instead of Ownable to Prevent Accidental Ownership Loss

Resolved

Path

erc20.sol/BSCToken.sol/TronToken.sol

Description

The contract inherits from OpenZeppelin's Ownable, which transfers ownership immediately in a single transaction. If the owner accidentally passes a wrong or unreachable address to transferOwnership, ownership is permanently lost with no recovery path. Given that the owner controls the mint function which has no supply cap losing ownership also means losing any future ability to manage token supply.

Recommendation

Replace Ownable with Ownable2Step.



Fix Floating Pragma to a Specific Compiler Version

Resolved

Path

erc20.sol/BSCToken.sol/TronToken.sol

Description

The contract uses a floating pragma `^0.8.27`, which allows compilation with any Solidity version $\geq 0.8.27$ $< 0.9.0$. This means the deployed bytecode could differ depending on which compiler version is used, potentially introducing subtle bugs or behavioral differences introduced in newer compiler versions. It also makes it harder to verify the exact compiler version used in production.

Recommendation

Lock the pragma to the exact version used during development



Functional Tests

Some of the tests performed are mentioned below:

- ✓ Deployment mints exactly 1,000,000,000 VIITUM tokens to the specified recipient address
- ✓ Deployment assigns the correct initialOwner as the contract owner
- ✓ transfer() correctly moves tokens between two EOA addresses
- ✓ batchTransfer() successfully distributes tokens to multiple recipients with correct amounts
- ✓ batchTransfer() reverts when recipients and amounts arrays have mismatched lengths
- ✓ batchTransfer() reverts when either array is empty
- ✓ batchTransfer() reverts when any recipient is the zero address
- ✓ batchTransfer() reverts when sender has insufficient balance for the total amount
- ✓ batchTransfer() reverts when any individual amount is zero
- ✓ mint() successfully mints tokens when called by the owner
- ✓ mint() reverts when called by a non-owner address
- ✓ burn() correctly reduces the caller's balance and total supply
- ✓ burnFrom() correctly burns tokens from an approved address
- ✓ approve() and transferFrom() work correctly within approved allowance
- ✓ transferFrom() reverts when transfer amount exceeds allowance
- ✓ permit() allows gasless approval via valid EIP-712 signature
- ✓ permit() reverts on expired deadline
- ✓ permit() reverts on invalid signature
- ✓ transferOwnership() correctly transfers ownership to a new address
- ✓ Token name returns VIITUM and symbol returns VIIT



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20 (OpenZeppelin)	Base Contract	Core token logic - balances, transfers, allowances	Correctly implements EIP-20 standard	If base has a bug, all token operations are affected
ERC20Burnable (OpenZeppelin)	Extension	Adds burn() and burnFrom() for token holders and approved callers	Correctly reduces supply and balances on burn	None well-audited, stable extension
ERC20Permit (OpenZeppelin)	Extension	Gasless approvals via EIP-712 signed messages	Correctly implements EIP-2612; ecrecover returns correct signer	Permit does not work for smart contract wallets (no EIP-1271); replay risk if chainId not enforced
Ownable (OpenZeppelin)	Base Contract	Single-step ownership access control for mint()	Owner address is valid, reachable, and competent	Single-step transfer risks permanent ownership loss; single key compromise enables unlimited minting



2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	constructor(address recipient, address initialOwner)	What this function can do Mint fixed initial supply ($1,000,000,000 \times 10^{18}$) to recipient; set initialOwner as owner; init EIP-712 domain "VIITUM" What this function cannot/should not do Run more than once; mint to address(0) (reverts ERC20InvalidReceiver); set owner to address(0) (reverts OwnableInvalidOwner) Main invariant(s) Initial supply minted exactly once; owner set once at deployment Access level Implicit (deployment only) Note recipient $\neq$ initialOwner allowed; supply NOT truly fixed because mint() exists
erc20.sol BSCToken.sol TronToken.sol	mint(address to, uint256 amount)	What this function can do Owner mints any amount to any non-zero address, raising total supply What this function cannot/should not do Be called by non-owner (OwnableUnauthorizedAccount); mint to address(0)



Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	batchTransfer (address[] recipients, uint256[] amounts)	Main invariant(s) Only owner can increase supply Access level onlyOwner Note No max-supply cap - inflation / centralization risk What this function can do Sender transfers to many recipients atomically; emits BatchTransferExecuted What this function cannot/should not do Empty/length-mismatched arrays (InvalidBatchTransfer); zero recipient (ERC20InvalidReceiver); zero amount (InvalidBatchTransfer); spend over balance (InsufficientBalanceForBatch) Main invariant(s) $\sum \text{amounts} \leq \text{sender balance}$; sender debited by exact total; supply unchanged Access level Public (any holder) Note Unbounded loop → gas/DoS with large arrays; always returns true
erc20.sol BSCToken.sol TronToken.sol	transfer(addr ess to, uint256 value)	What this function can do Move tokens from the caller's balance to another address. Update sender and recipient balances accordingly. What this function can do Send to address(0); exceed balance (ERC20InsufficientBalance)



Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	transferFrom(address from, address to, uint256 value)	Main invariant(s) Sender -value, recipient +value; supply unchanged Access level Public What this function can do Spender moves from's tokens via allowance What this function cannot/should not do Exceed allowance (ERC20InsufficientAllowance) or balance; send to address(0) Main invariant(s) Allowance -value (unless max, unchanged); balances adjusted; supply unchanged Access level Public (needs prior approval)
erc20.sol BSCToken.sol TronToken.sol	approve(addr ess spender, uint256 value)	What this function can do Set spender's allowance to value What this function cannot/should not do Approve address(0) spender (ERC20InvalidSpender) Main invariant(s) allowance[caller][spender] == value Access level Public Note Classic approve front-running race

Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	burn(uint256 value)	What this function can do Destroy caller's own tokens What this function cannot/should not do Burn more than balance Main invariant(s) Caller balance -value and supply -value Access level Public (self only)
erc20.sol BSCToken.sol TronToken.sol	burnFrom(address account, uint256 value)	What this function can do Spender burns account's tokens via allowance What this function cannot/should not do Exceed allowance or balance Main invariant(s) Account balance -value, supply -value, allowance -value Access level Public (needs allowance)
erc20.sol BSCToken.sol TronToken.sol	permit(owner, spender, value, deadline, v, r, s)	What this function can do Gasless approval via off-chain signature What this function cannot/should not do Expired deadline (ERC2612ExpiredSignature); invalid signer (ERC2612InvalidSigner); replay (nonce consumed) Main invariant(s) nonces[owner] +1; allowance set to value; signature matches owner Access level Public (anyone may submit a valid sig)

Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	transferOwnership(address newOwner)	What this function can do Transfer ownership to newOwner What this function cannot/should not do Set newOwner to address(0) (OwnableInvalidOwner); be called by non-owner Main invariant(s) owner() == newOwner after Access level onlyOwner
erc20.sol BSCToken.sol TronToken.sol	renounceOwnership()	What this function can do Set owner to address(0), permanently disabling all onlyOwner functions What this function cannot/should not do Be called by non-owner Main invariant(s) owner() == address(0); mint() permanently uncallable Access level onlyOwner Note Irreversible - effectively caps supply



Contract Name	Function	Category
erc20.sol BSCToken.sol TronToken.sol	name() · symbol() · decimals() · totalSupply() · balanceOf(ad dress) · allowance(ad dress,address) · owner() · nonces(adre ss) · DOMAIN_SEP ARATOR() · eip712Domain ()	What this function can do Return token/account/domain state (name "VIITUM", symbol "\$VIIT", decimals 18, balances, allowances, owner, permit nonce, EIP-712 domain) What this function cannot/should not do Modify any state Main invariant(s) Read-only; values reflect prior writes only Access level Public view (no restriction)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
erc20.sol BSCToken.sol TronToken.sol	ERC20 (self)	Tracks all VIITUM balances; initial 1,000,000,000 tokens distributed to recipient at deploy



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
erc20.sol BSCToken.sol TronToken.sol	constructor	–	VIITUM (1B minted to recipient)	Deployer (once)	Wrong recipient address = permanent loss of entire initial supply
erc20.sol BSCToken.sol TronToken.sol	mint()	–	VIITUM (new tokens created)	Owner only	Uncapped - owner can inflate supply arbitrarily; single key risk across all chains
erc20.sol BSCToken.sol TronToken.sol	burn()	VIITUM (destroyed)	–	Token holder	Irreversible; no recovery mechanism
erc20.sol BSCToken.sol TronToken.sol	burnFrom()	VIITUM (destroyed from from)	–	Approved spender	Requires allowance; allowance approval phishing risk
erc20.sol BSCToken.sol TronToken.sol	transfer()	–	VIITUM (to recipient)	Token holder	Standard transfer; no non-standard risk given fixed-decimal non-rebasing design
erc20.sol BSCToken.sol TronToken.sol	transferFrom()	–	VIITUM (to recipient)	Approved spender	Allowance manipulation; front-run risk on approve() → transferFrom()



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
erc20.sol BSCToken.sol TronToken.sol	batchTransfer()	–	VIITUM (to N recipients)	Any token holder	No batch size cap - gas limit DoS possible; atomicity failure on very large batches
erc20.sol BSCToken.sol TronToken.sol	permit()	–	Allowance granted	Anyone with valid sig	–

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of VIITUM. We performed our audit according to the procedure described above.

No critical issues in the VIITUM token, just 2 issues of Informational severity were found, which have been noted by the VIITUM team.

All of issues were resolved.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



VIITUM



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com